

Pedidos Veloz — Relatório Técnico e Prático

Do Docker Compose ao Kubernetes: implementação de referência com CI/CD e observabilidade
| Repositório: pasta **pedidos-veloz/** anexa a esta entrega

1. Visão geral do repositório

O repositório **pedidos-veloz/** contém a implementação completa da plataforma: quatro microsserviços em Node.js/Express (API Gateway, Serviço de Pedidos, Serviço de Pagamentos e Serviço de Estoque), banco PostgreSQL, ambiente local via Docker Compose, manifests de produção para Kubernetes, pipeline de CI/CD em GitHub Actions, stack de observabilidade (Prometheus, Grafana, Jaeger, OpenTelemetry Collector) e um esqueleto de Terraform para provisionar o cluster. A estrutura de pastas e as instruções de execução estão documentadas no README.md na raiz do repositório.

2. Ambiente local com Docker Compose

Todos os serviços sobem com um único comando: `docker compose up --build` (após copiar `.env.example` para `.env`). O arquivo `docker-compose.yml` define: uma rede dedicada em bridge (`pedidos-veloz-net`) isolando o tráfego entre os containers do projeto; um volume nomeado (`pgdata`) para persistir os dados do Postgres entre reinícios; variáveis de ambiente parametrizadas via `.env` (nunca hardcoded nos arquivos versionados); e healthchecks com `depends_on: condition: service_healthy`, garantindo que os serviços de aplicação só iniciem depois que o Postgres estiver pronto para aceitar conexões. Um segundo arquivo, `docker-compose.observability.yml`, funciona como overlay opcional (`docker compose -f docker-compose.yml -f docker-compose.observability.yml up`) e sobe Prometheus, Grafana, Jaeger e o OpenTelemetry Collector sem poluir o ambiente básico do dia a dia de desenvolvimento.

O fluxo de negócio testável localmente: uma requisição `POST /api/pedidos` no Gateway aciona o Serviço de Pedidos, que reserva o item no Serviço de Estoque, processa o pagamento no Serviço de Pagamentos e, só então, persiste o pedido como CONFIRMADO. Falha em qualquer etapa aciona compensação (ex.: libera o estoque reservado se o pagamento for recusado) — um padrão de saga simplificado, síncrono, adequado ao volume do MVP.

3. Containerização e versionamento

Cada serviço tem um Dockerfile multi-stage próprio: um estágio **builder** (baseado em `node:20-alpine`) instala apenas dependências de produção (`npm install --omit=dev`); um estágio **runtime**, também em Alpine, copia somente `node_modules` e o código-fonte necessário — sem ferramentas de build, gerenciador de pacotes ou cache do npm na imagem final. Boas práticas de segurança aplicadas em todas as imagens: usuário não-root dedicado (`appuser`, criado explicitamente, sem privilégios de sistema), HEALTHCHECK nativo do Docker apontando para o endpoint `/health` de cada serviço, imagem base Alpine (superfície de ataque e tamanho mínimos) e `.dockerignore` excluindo `node_modules`, `.env` e arquivos de controle de versão do contexto de build. O versionamento das imagens é feito pela pipeline de CI/CD: cada build publicado no GitHub Container Registry recebe a tag do SHA do commit além de `latest`, garantindo rastreabilidade exata entre código e artefato implantado.

4. Kubernetes — produção mínima

Os manifests em `k8s/` implementam: um **Namespace** dedicado com Pod Security Admission no nível `restricted` (exige `containers non-root`, sem escalonamento de privilégio e com `seccomp RuntimeDefault` em todos os Pods do namespace); um **ConfigMap** centralizando configuração não-sensível (`hosts` internos, `flags` de ambiente); um **Secret** de exemplo para credenciais do banco (nunca commitado em texto plano — o repositório inclui apenas `02-secrets.example.yaml`, com o arquivo real listado no `.gitignore`); **Deployments** para os quatro serviços de aplicação, cada um com

readinessProbe em /ready (inclui checagem de conexão com o banco, quando aplicável) e livenessProbe em /health; requests/limits de CPU e memória definidos para todos os containers; e um **StatefulSet** para o PostgreSQL, com PersistentVolumeClaim dedicado.

Segurança adicional: securityContext em nível de container (allowPrivilegeEscalation: false, readOnlyRootFilesystem: true, capabilities: drop [ALL]) e um conjunto de **NetworkPolicies** (08-networkpolicy.yaml) que nega todo tráfego por padrão (default-deny-all) e libera explicitamente apenas as conexões necessárias — por exemplo, o Postgres só aceita conexões dos Pods de Pedidos e Estoque, nunca do Gateway ou de fora do cluster. Essa postura de menor privilégio reduz o raio de impacto de um serviço comprometido.

5. CI/CD

O pipeline (.github/workflows/ci-cd.yml, GitHub Actions) roda em quatro estágios encadeados:

lint-test — Executa em matriz para os 4 serviços em paralelo: checagem sintática (node --check) e testes unitários (npm test, runner nativo do Node.js) sobre a lógica de validação de cada serviço, extraída em módulos puros testáveis sem I/O.

build — Builda a imagem Docker de cada serviço (sem publicar) e roda um scan de vulnerabilidades com Trivy (severidades CRITICAL/HIGH), hoje não-bloqueante para não travar o MVP, com recomendação de torná-lo bloqueante após estabilização.

publish — Apenas em push para main: publica as 4 imagens no GitHub Container Registry, autenticado via GITHUB_TOKEN (secret automático, sem necessidade de cadastro manual), com tags pelo SHA do commit e latest.

deploy — Protegido por um ambiente production do GitHub com aprovação manual obrigatória (gate humano antes de qualquer mudança em produção); aplica os manifests via kubectl usando o kubeconfig armazenado no secret KUBE_CONFIG_DATA; aguarda o rollout com kubectl rollout status e executa kubectl rollout undo automaticamente em caso de falha.

6. Observabilidade, deploy e escala

6.1 Métricas, logs e tracing

Métricas: todos os serviços expõem /metrics via prom-client, incluindo métricas padrão de runtime e métricas de negócio customizadas (pedidos_criados_total, pedidos_recusados_total{motivo}, pagamentos_processados_total{status}), coletadas pelo Prometheus e visualizáveis no Grafana. Logs: formato JSON estruturado escrito em stdout (services/*/src/logger.js), aderente ao princípio 12-Factor de logs como fluxo de eventos — nenhum serviço grava em arquivo local, delegando coleta e retenção à plataforma. Tracing distribuído: os quatro serviços são instrumentados com o SDK Node do OpenTelemetry (services/*/src/tracing.js), com instrumentação automática de HTTP/Express/pg, exportando spans via OTLP para um Collector que encaminha ao Jaeger. A propagação do contexto de trace (header W3C traceparent) acontece automaticamente entre as chamadas HTTP do Gateway → Pedidos → Pagamentos/Estoque, permitindo reconstruir no Jaeger a jornada completa de um pedido através dos quatro serviços — resolvendo diretamente o problema de baixa rastreabilidade de falhas relatado pela Loja Veloz.

6.2 Estratégia de deploy: Rolling Update

Adotou-se rolling update nativo do Kubernetes (maxSurge: 1, maxUnavailable: 0 em todos os Deployments), que sobe uma nova réplica antes de derrubar uma antiga, garantindo zero downtime sem exigir infraestrutura duplicada. Blue-Green foi descartado para o MVP por dobrar o custo de infraestrutura durante a transição; Canary foi descartado por exigir um controlador de tráfego mais sofisticado (Argo

Rollouts/Flagger ou service mesh) que a equipe não tem tempo de operar com segurança em 4 semanas. Rollback é automático via `kubectl rollout undo` disparado pela própria pipeline caso o rollout falhe.

6.3 Estratégia de escalabilidade: HPA

Horizontal Pod Autoscaler configurado para os quatro serviços com base em utilização de CPU, com limites diferenciados por criticidade observada: API Gateway (2–10 réplicas, 65% CPU) e Serviço de Estoque (2–8 réplicas, 70% CPU) concentram o maior volume de chamadas nos picos da campanha; Serviço de Pedidos (2–8 réplicas) e Pagamentos (2–6 réplicas, menor variação de carga) completam a política. VPA não foi adotado neste MVP: por reiniciar Pods para redimensionar recursos, é incompatível com um rolling update verdadeiramente zero-downtime — decisão alinhada ao requisito de reduzir indisponibilidade durante deploys.

7. Infraestrutura como código (Terraform)

O diretório `terraform/` provisiona, via `provider telemetry/kind`, um cluster Kubernetes local reproduzível (`kind`), parametrizado por variáveis (nome do cluster, número de nós worker, versão do Kubernetes). A escolha por um cluster local neste esqueleto — em vez de um provedor cloud gerenciado — prioriza custo zero e reprodutibilidade total para o MVP de 4 semanas: qualquer desenvolvedor ou o próprio CI provisiona um cluster idêntico em minutos, sem depender de credenciais de nuvem. Um módulo comentado no final de `main.tf` documenta o caminho de evolução para Amazon EKS usando o módulo oficial `terraform-aws-modules/eks/aws`, inspirado no case público de migração da iFood para o Amazon EKS. Os mesmos manifests em `k8s/` aplicam sem alteração tanto no `kind` local quanto em um cluster gerenciado real.

8. Referências do case pesquisado

Online Boutique (github.com/GoogleCloudPlatform/microservices-demo) — arquitetura de referência do Google Cloud usada como benchmark de decomposição em microsserviços e observabilidade com OpenTelemetry. Case público iFood + Amazon EKS (aws.amazon.com/pt/solutions/case-studies/ifood-eks-aws-case-study) — validou a direção de evolução de infraestrutura proposta no módulo Terraform, com resultados publicados de redução de tempo de escalonamento e de esforço operacional após a migração para Kubernetes gerenciado.

9. Links da entrega

Repositório: pasta `pedidos-veloz/` anexa a esta entrega (README.md com instruções completas de execução). Vídeo pitch (YouTube): *[inserir link após publicação — roteiro completo em docs/roteiro-video-pitch.md]*.